CS8.401 — Principles of Information Security

Programming Assignments — Team Project Plan

Team: Raj · Kanishk · Shubham · Shobhan · Swaraj

Language: Python 3.11 | Tests: pytest | Frontend: React + Flask API (PA#0)

---

1. Assignment Overview

This project implements the complete Minicrypt-to-MPC cryptographic reduction chain from scratch in Python 3.11, with no external cryptographic libraries. Every primitive must be built on top of your own prior implementation, traced from a One-Way Function (OWF) at the base all the way up to full 2-party Secure Multi-Party Computation.

The 21 Programming Assignments (PA#0 – PA#20) span four parts:

- Part I — Symmetric Cryptography (PA#1–6): OWF, PRG, PRF, Encryption, MACs
- Part II — Hashing & Data Integrity (PA#7–10): Merkle-Damgård, DLP hash, Birthday attack, HMAC
- Part III — Public-Key Cryptography (PA#11–17): DH, RSA, ElGamal, Signatures, CCA-PKC
- Part IV — MPC (PA#18–20): Oblivious Transfer, Secure AND, Full 2-party computation
- Part 0 — Minicrypt Clique Web Explorer (PA#0): React app built incrementally

Critical Rules:

- No external crypto libraries (no pycryptodome, cryptography, OpenSSL bindings, hashlib for HMAC, etc.). Only os.urandom and Python's built-in int (arbitrary precision) are permitted.
- Every bidirectional reduction (marked ✓) must implement BOTH A=>B and B=>A directions.
- Every PA must trace its entire dependency chain back to your own implementations.
- You MAY use: os.urandom, Python int arithmetic, pytest, flask/fastapi (web bridge only).

---

2. Team Assignments

The 21 PAs are divided by estimated complexity and code volume. Each member owns their PAs end-to-end: Python implementation, pytest tests, and the corresponding web app demo panel.

Raj — Cryptographic Foundations — OWF, PRG, PRF, Primality

PA#1 PA#2 PA#13	• Implement OWF: DLP f(x) = pow(g, x, p), AES-based f(k) = AES_k(0^128) XOR k, integer factoring f(p,q) = p*q • Implement PRG via HILL: iterative hard-core-bit construction G(x0) = b(x0)||b(x1)||... where x_{i+1} = f(x_i) • Implement GGM PRF tree: F_k(b1...bn) = G_bn(...G_b1(k)...) — depth-n binary tree traversal

- Implement AES-128 from scratch in Python (SubBytes, ShiftRows, MixColumns, AddRoundKey, KeySchedule)
- Bidirectional reductions: OWF<=>PRG (HILL + G is a OWF), PRG<=>PRF (GGM + G(s)=F_s(0)||F_s(1))
- Implement Miller-Rabin primality test (k=40 rounds) + gen_prime(bits) — shared by Shobhan
- Statistical test suite for PRG: NIST frequency (monobit), runs, serial — report p-values
- PRF distinguishing game: query on 100 random inputs, confirm no statistical difference vs random function
- Shared utility: mod_exp(base, exp, mod) via square-and-multiply (Python int is arbitrary precision — permitted)
- Web app panels: PRG live output viewer + GGM tree SVG visualizer

Kanishk — Symmetric Encryption & Modes of Operation

PA#3
PA#4
PA#6

- PA#3: CPA-Enc — Enc(k,m) = (r, F_k(r) XOR m), Dec(k,r,c) = F_k(r) XOR c; multi-block CTR extension with padding
- PA#3: IND-CPA game simulation (50-query adversary, advantage ~0); broken nonce-reuse variant demo
- PA#4: CBC mode — Enc(k, IV, M) and Dec(k, IV, C) for arbitrary-length messages, random IV per call
- PA#4: OFB mode — keystream independent of plaintext, pre-computable, encryption = decryption
- PA#4: Randomized CTR mode — sample nonce r internally, parallel block evaluation
- PA#4: Unified API: encrypt(mode, k, M) / decrypt(mode, k, C) routing to CBC/OFB/CTR
- PA#4: Attack demos — CBC IV-reuse (matching ciphertext blocks), OFB keystream-reuse (XOR to recover pt XOR)
- PA#6: CCA-Enc via Encrypt-then-MAC — uses PA#3 enc + PA#5 MAC; verify MAC before decrypting
- PA#6: Key separation demo; malleability attack on CPA-only (flip bit i => flipped plaintext bit i)
- PA#6: IND-CCA2 game simulation confirming negligible adversary advantage
- Web app panels: IND-CPA game + block cipher mode animator + malleability attack panel (left=CPA, right=CCA)

Shubham — MACs, Hashing, Birthday Attack & HMAC

PA#5
PA#7
PA#8
PA#9
PA#10

- PA#5: PRF-MAC (fixed-length): Mac(k,m) = F_k(m), Vrfy(k,m,t); uses Raj's PRF
- PA#5: CBC-MAC (variable-length): chain F_k over message blocks, final value = tag
- PA#5: EUF-CMA game — 50-query adversary cannot forge; length-extension attack on naive H(k||m)
- PA#5: MAC=>PRF backward direction demo: query MAC on uniform random inputs, pass PRF distinguishing test
- PA#7: Merkle-Damgard framework — MerkleDamgard(compress_fn, IV, block_size) generic class
- PA#7: MD-strengthening padding: append 1-bit, zero-pad, 64-bit big-endian length field
- PA#7: Dummy XOR compression for isolated testing; collision propagation demo
- PA#8: DLP CRHF — group setup (safe prime p=2q+1), h(x,y) = g^x * h_hat^y mod p

- PA#8: Plug DLP compression into PA#7 MD framework for full DLP_Hash(message) -> bytes
- PA#9: Naive birthday attack (dict-based) + Floyd cycle-finding (O(1) space) on any hash_fn
- PA#9: Empirical birthday curve: 100 trials, n in {8,10,12,14,16}, plot vs theoretical 1-e^{-k^2/2^n}
- PA#10: HMAC over DLP hash — HMAC_k(m) = H((k XOR opad) || H((k XOR ipad) || m))
- PA#10: Constant-time comparison (XOR all bytes); timing side-channel demo with naive early-exit
- PA#10: CRHF<=>HMAC<=>MAC bidirectional reductions; Encrypt-then-HMAC CCA-secure encryption
- PA#10: Length-extension attack on H(k||m) vs HMAC failure demo
- Web app panels: MAC forge demo + MD chain viewer + DLP hash + birthday attack live + length-extension panel

Shobhan — Public-Key Cryptography (Cryptomania)

PA#11
PA#12
PA#14
PA#15
PA#16
PA#17

- PA#11: DH key exchange — safe prime generation (uses Raj's Miller-Rabin), dh_alice/bob_step functions
- PA#11: MITM attack demo — Eve intercepts A,B, substitutes own values, holds both secrets
- PA#12: RSA keygen using Raj's gen_prime(); square-and-multiply mod exp (no pow() shortcut)
- PA#12: Extended Euclidean for modular inverse d = e^{-1} mod phi(N)
- PA#12: PKCS#1 v1.5 padding/unpadding; simplified Bleichenbacher padding oracle (toy 512-bit N)
- PA#12: Determinism attack demo — encrypt same message twice => identical ciphertexts
- PA#14: CRT solver crt(residues, moduli) via constructive formula + mod_inverse
- PA#14: CRT-based RSA decryption (Garner's algorithm) — benchmark ~4x speedup vs standard
- PA#14: Hastad broadcast attack — 3 recipients, e=3, Newton's integer cube root to recover m
- PA#14: Padding defeats attack demo using PKCS#1 v1.5
- PA#15: RSA digital signatures: sign(sk, m) = H(m)^d mod N; verify sigma^e mod N == H(m)
- PA#15: Hash-then-sign (mandatory); multiplicative forgery on raw RSA demo
- PA#16: ElGamal keygen/Enc/Dec; malleability attack (c1, 2*c2) => 2m; CPA game
- PA#17: Signcrypt = Encrypt-then-Sign (ElGamal Enc + RSA sign); Verify-then-Decrypt; IND-CCA2 game
- Web app panels: DH live exchange + RSA determinism + CRT Hastad visualizer + CCA malleability blocked

Swaraj — MPC (OT, Secure AND, Full 2-Party) + PA#0 Web App

PA#0
PA#18
PA#19
PA#20

- PA#0: React (Vite) app scaffold — foundation toggle (AES/DLP), two-column layout, routing table
- PA#0: Reduction routing: shortest-path chain among OWF/PRG/PRF/PRP/MAC/CRHF/HMAC

- PA#0: Bidirectional mode toggle (Forward A->B / Backward B->A), proof summary panel
- PA#0: Stub placeholders for all unimplemented PAs with correct PA# label — no crashes
- PA#0: Bridge: compile Python modules via Pyodide (in-browser) or expose via local Flask/FastAPI
- PA#18: 1-of-2 OT (Bellare-Micali) using Shobhan's ElGamal — OT_Receiver_Step1/2 + OT_Sender_Step
- PA#18: Receiver privacy demo (pk_{1-b} without trapdoor) + sender privacy demo (cannot decrypt C_{1-b})
- PA#19: Secure AND from OT — Alice sends (0, a) as OT messages, Bob uses choice bit b, gets a AND b
- PA#19: Secure XOR (free via additive secret sharing over Z_2) + Secure NOT (local flip)
- PA#19: Truth table test all 4 combos across 50 runs; transcript reveals nothing beyond output
- PA#20: Circuit class — DAG of AND/XOR/NOT gates, topological evaluation via Secure_Eval(circuit, x, y)
- PA#20: Millionaire's Problem (x > y over n-bit integers), Secure Equality (x == y), Secure Bit-Addition
- PA#20: End-to-end lineage: PA#20 AND => PA#19 OT => PA#16 ElGamal => PA#13 Miller-Rabin
- Web app panels: OT receiver step-by-step + AND gate 4-combo demo + Millionaire's live panel

---

3. Responsibility Matrix (All 21 PAs)

PA#	Topic	Owner	Depends On	Bidir?	Web Demo Panel
#0	Minicrypt Clique Web Explorer (React)	Swaraj	Stubs PA#1–10		Full app scaffold + routing table
#1	OWF + PRG (HILL construction)	Raj	—	✓	PRG live viewer + randomness test
#2	PRF via GGM Tree	Raj	#1 (own PRG)	✓	GGM tree SVG visualizer
#3	CPA-Secure Encryption	Kanishk	#2		IND-CPA game panel
#4	Modes of Operation (CBC/OFB/CTR)	Kanishk	#3		Block cipher mode animator
#5	Message Authentication Codes	Shubham	#2		MAC forge attempt demo
#6	CCA-Secure Encryption	Kanishk	#3 + #5		Malleability attack panel (CPA vs CCA)
#7	Merkle-Damgård Transform	Shubham	—		MD chain viewer

PA#	Topic	Owner	Depends On	Bidir?	Web Demo Panel
#8	DLP-Based CRHF	Shubham	#7		DLP hash live + collision hunt
#9	Birthday Attack (Collision Finding)	Shubham	#8		Live birthday attack chart
#10	HMAC + Encrypt-then-HMAC	Shubham	#8 + #3	✓	Length-extension vs HMAC panel
#11	Diffie-Hellman Key Exchange	Shobhan	#13		Live DH exchange + MITM demo
#12	Textbook RSA + PKCS#1 v1.5	Shobhan	#13		Determinism attack + PKCS comparison
#13	Miller-Rabin Primality	Raj	—		Primality tester with witness list
#14	CRT + Håstad Broadcast Attack	Shobhan	#12 + #13		Håstad broadcast visualizer
#15	Digital Signatures (RSA)	Shobhan	#12 or #16		Sign/verify live + forgery demo
#16	ElGamal Public-Key Cryptosystem	Shobhan	#11		ElGamal malleability panel
#17	CCA-Secure PKC (Signcrypt)	Shobhan	#15 + #16		CCA malleability blocked
#18	Oblivious Transfer (OT)	Swaraj	#12 or #16 (own!)		OT receiver step-by-step
#19	Secure AND Gate	Swaraj	#18		AND gate — all 4 input combos
#20	All 2-Party MPC (Yao/GMW)	Swaraj	#19 + #18		Millionaire's problem live panel

---

4. 8-Week Timeline

Week	Milestone	Work Items
Week 1	Setup + Foundations	ALL: agree on Python module interfaces (abstract base classes in interfaces/). Raj: Miller-Rabin (PA#13) + OWF (PA#1) + mod_exp shared utility. Swaraj: PA#0 React scaffold with stubs for all PAs.
Week 2	PRF/PRG + MAC	Raj: PA#2 GGM PRF + AES-128 from scratch. Kanishk: PA#3 CPA-Enc. Shubham: PA#5 MAC (PRF-MAC + CBC-MAC).
Week 3	Modes + Hashing	Kanishk: PA#4 CBC/OFB/CTR modes + attack demos. Shubham: PA#7 Merkle-Damgard + PA#8 DLP hash.
Week 4	CCA-Enc + Birthday + HMAC	Kanishk: PA#6 Encrypt-then-MAC (CCA-Enc). Shubham: PA#9 birthday attack + PA#10 HMAC + Encrypt-then-HMAC.

Week	Milestone	Work Items
Week 5	**DH + RSA + CRT**	Shobhan: PA#11 Diffie-Hellman + PA#12 RSA + PKCS#1 + PA#14 CRT + Hastad attack. Raj: supply gen_prime() to Shobhan.
Week 6	**Signatures + ElGamal + CCA-PKC**	Shobhan: PA#15 Digital Signatures + PA#16 ElGamal + PA#17 CCA-PKC (Signcrypt).
Week 7	**MPC Stack**	Swaraj: PA#18 OT (using Shobhan's ElGamal) + PA#19 Secure AND + PA#20 full 2-party MPC + all three circuits.
Week 8	**Integration + Web App**	Swaraj: fill PA#0 web app with real Python calls (Pyodide/Flask). All: integration tests, end-to-end lineage README, final review.

5. Python Project Architecture

Language: Python 3.11+. Tests: pytest. Web bridge: Flask or FastAPI (only for serving the React web app — not for crypto). No external crypto libraries. Permitted: os.urandom, Python built-in int (arbitrary precision integers — the one allowed exception per assignment spec).

5.1 Directory Structure

Module / File	Contents
src/foundations/	owf.py (DLP/AES/factor), prg.py (HILL), aes128.py (from scratch)
src/prf/	ggm_prf.py (GGM tree using PRG), aes_prf.py (AES as PRF)
src/prp/	luby_rackoff.py (3-round Feistel PRP from PRF)
src/enc/	cpa_enc.py, cbc.py, ofb.py, ctr.py, cca_enc.py
src/mac/	prf_mac.py, cbc_mac.py, hmac_impl.py (no hashlib!)
src/hash/	merkle_damgard.py (generic), dlp_hash.py (DLP compression + MD)
src/attack/	birthday.py (naive + Floyd), hastad.py, bleichenbacher.py, length_ext.py
src/primality/	miller_rabin.py, prime_gen.py (shared utility — Raj writes)
src/dh/	dh.py (key exchange, MITM demo)
src/rsa/	rsa.py (keygen/enc/dec), pkcs15.py, crt_rsa.py, garner.py
src/elgamal/	elgamal.py (keygen/enc/dec, malleability demo)
src/sig/	rsa_sig.py (hash-then-sign), elgamal_sig.py
src/pke/	signcrypt.py (Encrypt-then-Sign CCA-PKC)
src/mpc/	ot.py (Bellare-Micali), secure_and.py, circuit.py, millionaire.py
src/utils/	random_utils.py (os.urandom wrapper), mod_exp.py, ext_gcd.py, mod_inv.py, int_root.py
interfaces/	Abstract base classes: OWF, PRG, PRF, MAC, SymEnc, Hash, PKC, OT — define interfaces Week 1
tests/	test_pa1.py ... test_pa20.py — pytest, one file per PA
web/	React app (Vite) — Swaraj; Python called via Pyodide or local Flask/FastAPI API
api/	Flask/FastAPI server exposing Python PA implementations as REST endpoints for web app
requirements.txt	pytest, flask (web bridge only) — NO crypto libs (no pycryptodome, cryptography, etc.)

5.2 Abstract Interfaces (interfaces/*.py)

Define these as Python abstract base classes (ABC) in Week 1. Every implementation must subclass the relevant interface. This lets members work in parallel without breaking each other.

Interface	Key Method Signatures
OWF	`def evaluate(self, x: bytes) -> bytes: ...`
PRG	`def expand(self, seed: bytes, out_len: int) -> bytes: ...`
PRF	`def evaluate(self, key: bytes, x: bytes) -> bytes: ...`
MAC	`def tag(self, k: bytes, m: bytes) -> bytes: ... def verify(self, k: bytes, m: bytes, t: bytes) -> bool: ...`
SymEnc	`def encrypt(self, k: bytes, m: bytes) -> tuple[bytes, bytes]: ... def decrypt(self, k: bytes, ct: tuple) -> bytes: ...`
Hash	`def hash(self, msg: bytes) -> bytes: ...`
PKC	`def keygen(self, bits: int) -> tuple[PublicKey, PrivateKey]: ... def encrypt(self, pk, m: bytes) -> bytes: ... def decrypt(self, sk, c: bytes) -> bytes: ...`
OT	`def receiver_step1(self, choice_bit: int) -> tuple[PublicKey, PublicKey, object]: ... def sender_step(self, pk0, pk1, m0: bytes, m1: bytes) -> tuple[bytes, bytes]: ... def receiver_step2(self, state, c0: bytes, c1: bytes) -> bytes: ...`

5.3 Python-Specific Notes

- Big integers: Python's built-in int is arbitrary precision — use directly. pow(base, exp, mod) is permitted for modular exponentiation as it's a built-in integer operation, not a crypto library. However, you must implement your own square-and-multiply to show understanding.
- Randomness: use os.urandom(n) everywhere. Never use random.random(), secrets, or any other source.
- AES-128: implement SubBytes (S-box), ShiftRows, MixColumns (GF(2^8) arithmetic), AddRoundKey, KeySchedule fully from scratch in aes128.py.
- Byte manipulation: use Python bytes/bytearray; XOR via bytes(a ^ b for a, b in zip(x, y)).
- No hashlib: not even for HMAC or SHA — PA#10 HMAC must use your own DLP hash from PA#8.
- Web bridge: Flask or FastAPI is permitted ONLY to serve Python implementations as HTTP endpoints for the React app. All crypto logic stays in src/.

---

6. Full Reduction Chain (How Everything Connects)

Every arrow below must be traceable as actual Python function calls. Bidirectional arrows (<=>) require both directions implemented and demonstrated.

Layer 1 — Minicrypt Core:

- OWF (PA#1) <=> PRG (PA#1) <=> PRF (PA#2, GGM) <=> PRP (Luby-Rackoff)
- PRF => MAC (PA#5) | PRF + MAC => CCA-Enc (PA#6) | PRF => CPA-Enc (PA#3)
- CRHF (PA#7+8) <=> HMAC (PA#10) <=> MAC (PA#5) [Birthday floor: PA#9]

Layer 2 — Cryptomania:

- OWF => DH (PA#11) => RSA (PA#12) => ElGamal (PA#16)
- ElGamal/RSA => Signatures (PA#15) => CCA-PKC (PA#17)

Layer 3 — MPC:

- CCA-PKC (PA#17) => OT (PA#18) => Secure AND (PA#19) => All 2-Party MPC (PA#20)

End-to-end call stack for one AND gate in PA#20:

- secure_and(a, b) [PA#19]
  - calls ot.receiver_step1(b) + ot.sender_step(pk0, pk1, 0, a) [PA#18]
  - calls elgamal.encrypt(pk, m) [PA#16]
  - calls dh.generate_group() [PA#11]
  - calls prime_gen.gen_prime(bits) [PA#13]
  - calls miller_rabin.is_prime(n) [PA#13]
  - uses pow(a, d, n) [Python built-in int — permitted]

This full chain must be documentable in your README — no library shortcuts anywhere.

---

7. Testing Strategy

Per-PA pytest Files (tests/test_paN.py)

- Correctness: decrypt(encrypt(m)) == m, verify(mac(m)) == True, etc.
- Game simulations: IND-CPA, IND-CCA2, EUF-CMA — run 100 rounds, adversary advantage < 0.1.
- Attack demos: broken schemes must demonstrably fail when attacked (nonce reuse, deterministic RSA, etc.).
- Bidirectional: for each PA marked ✓, test BOTH directions produce correct output.

Shared / Integration Tests

- PRG statistical: pytest collects p-values from NIST frequency, runs, serial tests — must all pass at p > 0.01.
- Birthday empirical: 100 trials per n in {8,10,12,14,16} bits, ratio vs 2^(n/2) within 2×.
- MPC lineage: assert that executing PA#20 Millionaire triggers PA#18 OT which triggers PA#16 ElGamal (use mock/spy if needed).
- Run all tests: pytest tests/ -v from project root.

---

8. Team Coordination

Week 1 — Interface Agreement (Critical)

Every member must meet in Week 1 to agree on and commit interfaces/owf.py, interfaces/prf.py, interfaces/mac.py, etc. before writing any implementation. Stubs that implement the interface and return fixed test vectors are enough to unblock all other members.

Shared Utilities — Raj Delivers Week 1

- src/utils/random_utils.py — os.urandom wrapper with generate(n: int) -> bytes
- src/utils/mod_exp.py — square_and_multiply(base, exp, mod) implemented manually
- src/utils/ext_gcd.py — extended_gcd(a, b) -> (gcd, x, y)
- src/primality/miller_rabin.py + prime_gen.py — gen_prime(bits) shared by Shobhan's PKC

Dependency Unlock Order

- Shobhan is blocked on Raj's gen_prime() — Raj stubs this first with a known prime list for small bits.
- Kanishk is blocked on Raj's PRF — Raj stubs PRF(k, x) = XOR(k, x) so Kanishk can start PA#3.
- Swaraj (PA#18 OT) is blocked on Shobhan's ElGamal — Shobhan provides stub returning fixed group element.
- Shubham's PA#10 HMAC is blocked on PA#8 DLP hash (own dependency) and PA#3 CPA-enc (Kanishk).

Code Review Checkpoints

- Week 2: Raj demos PRF/GGM, Kanishk demos CPA-Enc — 15 min each.
- Week 4: Shubham demos HMAC + birthday, Kanishk demos CCA-Enc.
- Week 6: Shobhan demos full PKC stack (DH through CCA-PKC).
- Week 8: Swaraj demos MPC + web app. All: final integration review + README check.

---

9. PA#0 Web App — Swaraj Leads, All Contribute

The React web app (Vite) is built incrementally. Swaraj owns the scaffold; each member contributes demo panel components as they finish their PA.

Python-to-Browser Bridge

- Option A (recommended): Local Flask/FastAPI server (api/) exposes Python implementations as REST endpoints. React calls these via fetch(). Simple and debuggable.
- Option B: Compile Python to WebAssembly via Pyodide (in-browser Python runtime). No server needed but more complex setup.

Each Member Delivers (alongside their PA)

- Raj: GGM tree SVG visualizer component + PRG live output viewer + Miller-Rabin witness table
- Kanishk: IND-CPA game component + CBC/OFB/CTR mode animator + malleability attack panel
- Shubham: MD chain viewer + birthday attack live chart + HMAC length-extension panel
- Shobhan: DH exchange panel + RSA determinism demo + Håstad visualizer + CCA-PKC panel
- Swaraj: PA#0 scaffold + routing table + bidirectional toggle + OT panel + Millionaire's panel

---

10. Summary — Who Does What

Member	PAs Owned	Shared Utility	Core Deliverable
Raj	**PA#1, PA#2, PA#13**	random_utils.py, mod_exp.py, ext_gcd.py, miller_rabin.py — deliver Week 1	Cryptographic Foundations — OWF, PRG, PRF, Primality

Member	PAs Owned	Shared Utility	Core Deliverable
Kanishk	**PA#3, PA#4, PA#6**	Own tests + demo panel components	Symmetric Encryption & Modes of Operation
Shubham	**PA#5, PA#7, PA#8, PA#9, PA#10**	Own tests + demo panel components	MACs, Hashing, Birthday Attack & HMAC
Shobhan	**PA#11, PA#12, PA#14, PA#15, PA#16, PA#17**	Own tests + demo panel components	Public-Key Cryptography (Cryptomania)
Swaraj	**PA#0, PA#18, PA#19, PA#20**	Web app scaffold + Flask API bridge	MPC (OT, Secure AND, Full 2-Party) + PA#0 Web App

Build the chain — one reduction at a time!